

RELATÓRIO 11 — CONSOLIDADO

M.I.N.D. — Muñdji Intelligent Neural Developer
Estado completo do projecto a 5 de Agosto de 2026

Campo	Valor
Substitui	<i>Relatorio_MIND_2026-08-03_08_consolidado.pdf</i> , que não conhecia o modo Anthropic nem a tentativa de piloto
Entregas	13 commits de trabalho + 11 relatórios
Código	19 módulos em <i>agent/</i> , 5535 linhas
Suite	351 testes passam · 4 saltados
Branch	<i>claude/mind-prompt-implementation-9762wh</i> · PR #1 · HEAD 8d58276
Estado numa frase	Arquitectura completa e exercitada. Nunca correu com um LLM verdadeiro.

1. O que o MIND é

Um sistema multi-agente de geração autónoma de código sobre LangGraph. Um **CORTEX** orquestra (com persona JARVIS), um **CEREBELLUM** audita sem persona, e **seis NEURONS** executam partes distintas do mesmo ficheiro. O ciclo repete-se até o código atingir o limiar de aprovação ou esgotar as iterações.

As cinco decisões que definem o sistema, e que sobreviveram a todas as revisões:

- **Marcadores *[NEURON_N:linguagem]*.** A linguagem lê-se do marcador, nunca se infere. É o que permite um ficheiro multi-linguagem sem ambiguidade.
- **Contrato de interface com diff real.** A heurística (marcador próprio presente, marcadores alheios ausentes) é filtro rápido; a autoridade é o diff verdadeiro das secções alheias, reconstruído *por secções* e não por concatenação de texto.
- **Validação cruzada.** Dois relatórios independentes; divergência acima de 15pp força uma 3ª ronda antes de decidir.
- **Activação dinâmica.** Todos os NEURONS na 1ª passagem; a partir da 2ª iteração só os visados.
- **SYNAPSE DB local.** SQLite em WAL. Tudo local — os dados operacionais nunca saem do teu hardware. Mantém-se intacto mesmo com inferência remota.

2. Cronologia das 13 entregas

#	Commit	Entrega
1	<i>ac25c16</i>	Base do MIND: CORTEX, CEREBELLUM, NEURONS, SYNAPSE DB, sandbox multi-linguagem, sanitizador, backup, versionamento git, CLI
2	<i>fdcb706</i>	Camada HIPPOCAMPUS (ML consultivo): assimétrica — apoia rejeição automática, nunca aprovação automática
3	<i>4f51da6</i>	Suite de testes (151 testes iniciais)
4	<i>7c6d807</i>	Fecho de lacunas: diff de contrato real, limpeza git verdadeira, decisão LangGraph por teste técnico, esquema JSON dos relatórios
5	<i>deec79d</i>	Router para HuggingFace em GPU alugada: 4 modos, token por componente

#	Commit	Entrega
6	06a57a6	Sandbox de testes evolutiva: biblioteca que cresce entre ciclos, rigor escalonado, percentagem medida em vez de estimada
7	a143abf	Ferramenta de piloto: <i>verificar</i> e <i>piloto</i> , medição e exportação CSV
8	de9e0a1	Ensaio do runbook sem GPU; discrepância do <i>ml-status</i> corrigida
9	a71e5f8	Modo anthropic — 5º modo do router, ligação directa à API
10	8789d7c	Corpo do erro reportado — o <i>verificar</i> deixava cair a razão real da falha
11	0b0967d	Guarda de truncagem — <i>stop_reason=max_tokens</i> deixa de passar por sucesso
12	43068f2	Medição da conformidade corrigida — reportava 77,8% quando a verdade era 0%
13	8d58276	Relatório 10

3. Os defeitos reais encontrados e corrigidos

Vale a pena listá-los porque são a medida da diferença entre «escrito» e «a funcionar». Todos foram encontrados a exercitar o sistema, nenhum a lê-lo.

Defeito	Como se manifestava	Porque importava
Versionamento git decorativo	O MIND nunca escrevia no workspace durante as iterações, logo <i>commit_iteration</i> não tinha o que comitar	A funcionalidade existia no papel e não no facto
Falhas de modelo sem rasto	Registadas com <i>cycle_id=0</i> , violando a foreign key; o <i>except</i> engolia o erro	As falhas que mais interessa investigar eram as que desapareciam
Limpeza git não limpava	As <i>tags</i> antigas mantinham os commits temporários alcançáveis	O <i>gc</i> corria e não podava nada
Falso positivo no diff de contrato	Reconstrução por concatenação duplicava o preâmbulo	NEURONS honestos eram acusados de violar o contrato
<i>rustc</i> presente sem toolchain	<i>shutil.which</i> dava positivo; a compilação falhava	A sandbox dizia suportar Rust e não suportava
Corpo do erro descartado	Um 400 por falta de saldo aparecia como «Client error 400» e um link para a Mozilla	A ferramenta cujo único trabalho é diagnosticar não diagnosticava
Truncagem silenciosa	<i>stop_reason=max_tokens</i> vinha com 200 e texto cortado	Código incompleto seria reprovado como código mau
Conformidade JSON mal medida	Denominador contava chamadas sem esquema; numerador ignorava 1 dos sítios	É critério de escolha de modelo. 77,8% quando a verdade era 0%

4. O que está provado e o que não está

Esta é a secção que interessa, e a linha divisória é sempre a mesma: **está provado tudo o que depende do código do MIND; não está provado nada que dependa do que um modelo verdadeiro responde.**

PROVADO	Como
Os 5 modos do router formatam correctamente	351 testes; e o modo <i>anthropic</i> contra a API real — o 400 de saldo só se alcança depois de passar autenticação, versão e validação do corpo
Todos os pontos de chamada produzem pedidos válidos	Ciclos completos contra um servidor de ensaio que rejeita com 400 qualquer desvio ao contrato
O ciclo corre de ponta a ponta	Fases 1-2-3, sandbox, SYNAPSE DB, versionamento, limpeza de temporários
A sandbox evolutiva escala o rigor	3 iterações de média contra 1 sem ela; testes gerados, executados e contados
A conformidade JSON é medida bem	Verificada nos dois extremos: 0% e 100%
As guardas disparam	Truncagem, contrato de interface, circuit breaker, retry com backoff

NÃO PROVADO	Porquê
Que os modelos geram código que compila	As respostas do ensaio são fixas e escritas por mim
Quantas iterações um ciclo real precisa	As medições do ensaio são artefactos das respostas fixas
Quanto demora e quanto custa	As durações medidas (0,6s) são latência de um servidor local
Se 4096 tokens chegam	A guarda existe; se dispara na prática, não se sabe
Se os modelos respeitam o esquema JSON	Sabe-se medir. Não se sabe o valor
Que o MIND está 100% funcional	Exige tudo o que está acima

5. O que não fiz, e porquê

Não fiz	Razão
Contornar a falta de saldo da API	Não há maneira legítima, e procurá-la seria a gambiarra proibida
Subir o <i>max_tokens</i> de 4096	É o valor da especificação. Sei que os modelos suportam 64k-128k, mas não sei se 4096 estorva. Mudá-lo sem evidência é o ajuste às cegas
Preencher <i>config/neuron_specialties.yaml</i>	As especialidades dependem dos modelos, e os modelos escolhem-se depois do piloto
Preencher as fichas de <i>model_selection_criteria.md</i>	Pedem resultados de teste-piloto que ainda não existem
Declarar o sistema validado	Seria falso
Usar Redis	Com um servidor de inferência, uma fila não traz paralelismo real. Documentado como caminho futuro, não como falta
Construir GUI	Fora de âmbito por indicação explícita
<i>streaming, prompt caching, extended thinking</i>	Fora do âmbito da entrega do modo anthropic. O <i>thinking</i> mudaria o que o piloto mede — ligá-lo antes da linha de referência estragaria a medição

6. O que poderia fazer

- **Registar *usage* (tokens) por componente na SYNAPSE DB.** A resposta da API já o traz. Daria custo real por ciclo e por componente — o número que falta ao critério de selecção de modelos.
- **Prompt caching nas personas e instruções JSON.** São a parte constante de todas as chamadas. Com custo por token, é o ganho mais directo que existe.
- **Modos diferentes por componente.** Hoje *MODEL_MODE* é global. O token já é por componente, portanto misturar Anthropic no CORTEX com GPU alugada nos NEURONS é arquitecturalmente trivial.
- **Fine-tuning a partir da SYNAPSE DB.** O *export* já produz JSONL por componente. Falta volume de ciclos reais.
- **Activar o HIPPOCAMPUS.** Está implementado e desligado. Precisa de 100 amostras reais para valer alguma coisa.

7. O que vou fazer a seguir

#	Passo	Depende de
1	Correr as quatro passagens com modelos reais. É o único passo que converte «não provado» em «provado».	Saldo na conta da API.
2	Medir truncagem, conformidade, iterações, duração e custo.	Passo 1
3	Registar <i>usage</i> por componente, para haver custo real.	Passo 1
4	Preencher as fichas de modelo com resultados medidos.	Passo 2
5	Só então <i>neuron_specialties.yaml</i> .	Passo 4

Onde estamos, em duas frases: o MIND está inteiro, exercitado e mais fiável do que em qualquer ponto anterior — 351 testes, oito defeitos reais corrigidos, cinco modos de serviço. E continua sem nunca ter falado com um modelo verdadeiro, que é a única coisa que separa «construído» de «funciona».

O passo 1 já não depende de alugar GPU nem de escolher modelos. Depende de um valor de configuração e de saldo — que é uma decisão tua, não uma dependência técnica.